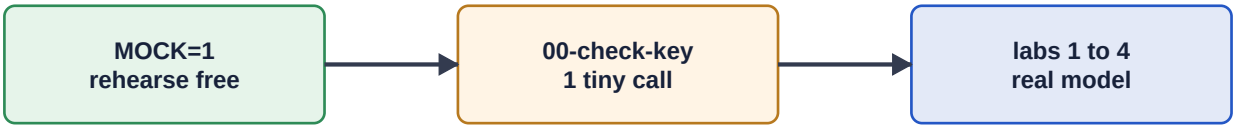
GUIDE 3 OF 3 FOR THE petri-flow REPOSITORY

PetriFlow with your own OpenAI key

Should you do this on a company laptop, and how. Four labs where a real model drives the safety gate on localhost, each with a free rehearsal mode, full source you can paste, and the output you should see.

Verified 9 October 2026: mock mode end to end, real mode up to the API (401 with a dummy key).



WHAT IS INSIDE

Part 1 Can you? The policy questions, what data leaves the laptop, cost
Part 2 What in the repo needs a key (and why it is Anthropic, not OpenAI)
Part 3 Setup: provider package, key in the shell, scratch folder, the mock switch
Part 4 Lab 0: prove the key works with one tiny call
Part 5 Labs 1 to 4: file agent, deploy agent with approvals, localhost server, engine decisions
Part 6 Using an Anthropic key with the repo's own examples instead
Part 7 Troubleshooting, cost control, cleanup, cheat sheet
Appendix The full source of every lab file, as paste-ready commands

Companions: 'PetriFlow, explained from zero' (concepts) and 'PetriFlow, run it yourself' (tests and workflows).

Contents

Contents 2

Part 1. Can you do this on a company laptop? 3

1.1 The questions, in plain words 3

1.2 What leaves the laptop 3

1.3 What it costs 4

Part 2. What in the repo needs a key 5

Part 3. Setup 6

Part 4. Lab 0: prove the key works 8

Part 5. The four labs 9

Lab 1: a file-cleanup agent 9

Lab 2: a deployment agent with human approval 10

Lab 3: a localhost server where history is the memory 11

Lab 4: the model as the engine's decision maker 12

Part 6. If you have an Anthropic key instead 14

Part 7. Troubleshooting, cost control, cleanup 15

7.1 Problems and fixes 15

7.2 Cost control 15

7.3 Cleanup 15

7.4 Cheat sheet 16

Appendix. The lab files, paste-ready 17

labs/openai/_model.ts 17

labs/openai/00-check-key.ts 18

labs/openai/01-file-agent.ts 19

labs/openai/02-deploy-agent.ts 20

labs/openai/03-server.ts 22

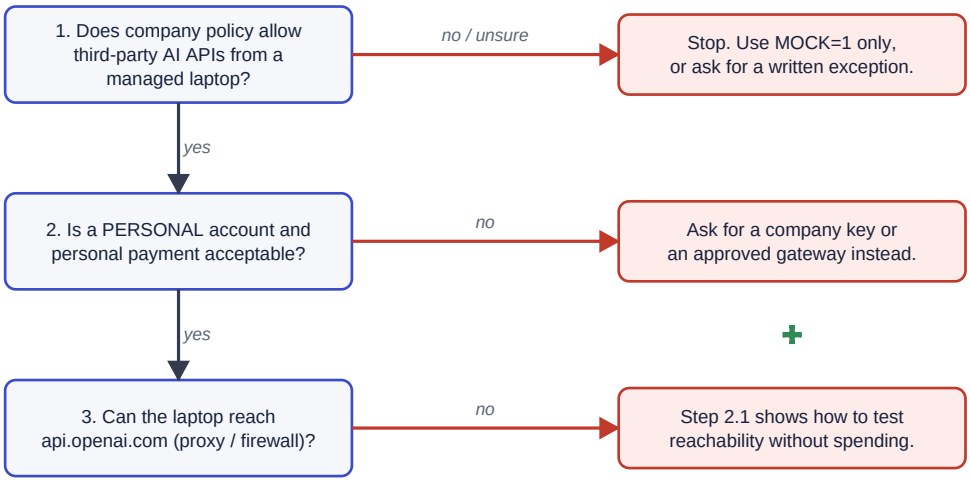
labs/openai/03-client.ts 24

labs/openai/04-decision-provider.ts 24

labs/openai/README.md 25

Part 1. Can you do this on a company laptop?

Nobody outside your company can answer that for you. What this guide can do is name the exact questions, show what the labs actually send over the network, and give you a way to do every lab with **zero** external calls while you wait for an answer.



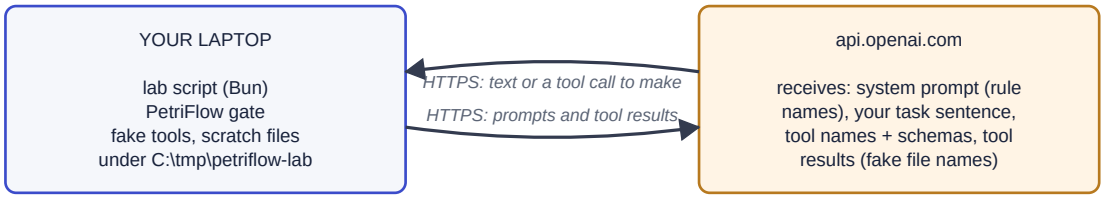
Three yes answers = proceed with the real key. Anything else = run the labs with MOCK=1, which needs no key and sends nothing anywhere.

The three questions to settle before exporting a real key on a managed laptop.

1.1 The questions, in plain words

- **Acceptable use.** Most companies have an AI or SaaS policy. Typical rules: no unapproved AI services from managed devices, no personal accounts for work activity, no company data in external models. Learning a public open-source repo is usually not 'work data', but the device is still managed. Ask your manager or IT in writing. Keep the reply.
- **Whose key, whose money.** A personal OpenAI key means personal billing. If the company later asks what you sent, you want a clean answer: synthetic prompts from a public repo, nothing else. If they offer a company key or an internal gateway, prefer that and set OPENAI_BASE_URL accordingly (see 7.1).
- **Network.** Corporate proxies often block or inspect api.openai.com. Step 2.1 tests this with a *dummy* key: a '401 Incorrect API key' answer proves the path is open without spending anything. A timeout or 'fetch failed' means blocked.

1.2 What leaves the laptop



Nothing from your company's code or documents is in these labs. The only data that leaves is what the scripts print on screen.

Data flow for every lab in this guide. The tools are fakes or act on a scratch folder.

Concretely, each request to OpenAI contains: a short system prompt that lists the PetriFlow rules (strings like 'require backup before delete'), your one-line task, the names and JSON schemas of the fake tools, and the results those tools returned (for example `{ok: true, deleted: 'temp.log'}`). Lab 1 touches real files, but only inside `C:\tmp\petriflow-lab`, which the script creates. Labs 2, 3 and 4 use simulated tools and never write outside the repo.

1.3 What it costs

Each lab makes between 1 and about 12 model calls with short prompts. With gpt-4o-mini a complete run of all four labs is well under one US cent at the prices published in 2026; check the current price list yourself before you run. Every lab caps the number of model steps (stopWhen), so a confused model cannot loop forever. Lab 0 prints token counts so you can see the real numbers.

> **NOTE Rules of hygiene for a personal key on a work machine**
>
> Export the key only in the Git Bash window you are using (`export OPENAI_API_KEY=...`), never in .bashrc, never in a file inside the repo, never in a ticket or chat. Set a monthly spending limit on your OpenAI account first. When you are done, close the window and revoke the key on the OpenAI dashboard. Run `git status` before any commit so a key cannot slip in.

> **IN SIMPLE WORDS If the answer is no, or not yet**
>
> Run every lab with `MOCK=1`. A scripted model stands in for OpenAI, the gate and tools run exactly the same code, and nothing leaves the laptop. You see every block, every approval and every replay. Only the model's 'thinking' is canned.

Part 2. What in the repo needs a key

Short version: **no test needs a key**, and everything that does need one is written for **Anthropic**, not OpenAI. The only OpenAI code in the repo is a snippet in a tutorial on the docs site. That is why this guide ships its own OpenAI versions under `labs/openai/`.

File	Provider	Needs	What it demonstrates
examples/01-file-management/agent.ts	Anthropic	ANTHROPIC_API_KEY	require backup before delete, block rm, real files under /tmp/project
examples/02-deployment/agent.ts	Anthropic	ANTHROPIC_API_KEY	lint -> test -> deploy, auto-approved human gate, limit 2. Its 'test' tool runs the whole repo test suite.
examples/03-discord-bot/agent.ts	Anthropic	ANTHROPIC_API_KEY + Discord bot token + guild id	dot-notation actions, rate limit
examples/04-devops-assistant/agent.ts	Anthropic	ANTHROPIC + Brave + Slack + email keys	13 tools, 10 rules, terminal y/n approvals
workflows/simple-agent/run-llm.ts	Anthropic	ANTHROPIC_API_KEY	LLM as the engine's decision provider
comparisons/openclaw-safety/src/run.ts	Anthropic	ANTHROPIC_API_KEY	interactive tool-approval scenario with LLM choices
site/.../tutorial-server-integration.astro	OpenAI (snippet)	OPENAI_API_KEY + hono	Hono server, fresh session per request. Not runnable from the repo as is.
labs/openai/* (this guide)	OpenAI	OPENAI_API_KEY, or MOCK=1	The same four ideas, ported to OpenAI, Windows-safe, with mock mode

LOOPHOLE Two things that bite on Windows even with a valid key

The repo's agent.ts files load their .rules file with `new URL(...).pathname`, which yields '/C:/...' on Windows and fails with ENOENT. The lab scripts avoid this by compiling rules inline. Example 01 also writes to /tmp/project, which is C:\tmp\project for Bun. The labs use C:\tmp\petriflow-lab and create it themselves.

Part 3. Setup

Prerequisites: Parts 1 and 2 of the lab guide are done (clone, bun install, bun run build). All commands run from the repo root in Git Bash.

Step 3.1 Add the OpenAI provider for the Vercel AI SDK

```
bun add @ai-sdk/openai@^3
grep -n ai-sdk package.json
```

Expected output

```
installed @ai-sdk/openai@3.0.124
 9 packages installed [10.16s]
21:    "@ai-sdk/openai": "^3",
```

WHAT IS EXECUTING

Bun adds one dependency line to the root package.json and updates bun.lock. The provider package is a thin HTTP client that speaks OpenAI's API and implements the AI SDK's model interface, the same interface the repo's Anthropic examples use. Version 3 matches ai@6 and @ai-sdk/anthropic@3 already in the repo. Revert later with `git checkout -- package.json bun.lock`.

Step 3.2 Create the lab files

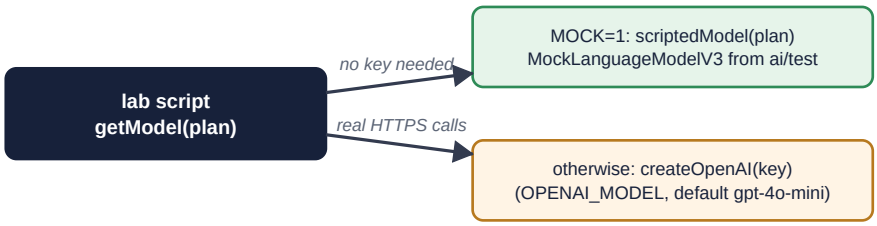
Create the folder and paste the seven files from the Appendix (one heredoc command each). Or copy the `labs/openai` folder from the machine where this guide was produced.

```
mkdir -p labs/openai
# then paste each 'cat > labs/openai/... <<'EOF'' block from the Appendix
ls labs/openai
```

Expected output

```
00-check-key.ts  01-file-agent.ts  02-deploy-agent.ts  03-client.ts  03-server.ts  04-decisi
    on-provider.ts  README.md  _model.ts
```

Step 3.3 The mock switch, and how real mode is selected



Everything after the model (gate, tools, replay, scheduler) is identical in both modes. The mock only replaces the LLM's decisions.

One helper, _model.ts, decides which model the labs talk to.

WHAT IS EXECUTING

`getModel(plan)` in _model.ts returns either a real OpenAI model or a `MockLanguageModelV3` from the AI SDK's own test utilities. The mock answers each model call with the next step of a plan the lab provides: a tool call with arguments, or a final text. Because the AI SDK, the gate, the wrapped tools and the replay logic do not know which model they are talking to, mock mode exercises 100 percent of the PetriFlow code path.

Step 3.4 Rehearse everything for free first

```
MOCK=1 bun run labs/openai/01-file-agent.ts
MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
MOCK=1 bun run labs/openai/04-decision-provider.ts
```

Each should end with exit code 0 and the outputs shown in Part 5. If they do, the only remaining variable is the key.

Step 3.5 Put the key in the shell (only when Part 1 is settled)

```
export OPENAI_API_KEY=sk-...paste your key...
export OPENAI_MODEL=gpt-4o-mini        # optional, this is the default
echo ${OPENAI_API_KEY:0:7}...            # shows sk-proj or sk-, never the \
  whole key
```

The variable lives only in this Git Bash window. Open a second window for the server lab and export it there too.

Part 4. Lab 0: prove the key works

Step 4.1 Without spending: reachability check with a dummy key

```
OPENAI_API_KEY=sk-test-dummy bun run labs/openai/00-check-key.ts
```

Expected output (this is the GOOD result for a dummy key)

```
key: sk-test...ummy  model: gpt-4o-mini

ERROR: Incorrect API key provided: sk-test-**********ummy. You can find your API key at ...
=> The key is wrong or revoked. Check OPENAI_API_KEY.
```

WHAT IS EXECUTING

The script builds a real OpenAI model with the dummy key and asks for the word OK. OpenAI rejects the key with HTTP 401 *after* the request crossed your proxy and firewall. A 401 therefore proves network reachability at no cost. If instead you see 'fetch failed', ENOTFOUND or a timeout, the network is blocked; set HTTPS_PROXY (lab guide step 1.1) or stop here and use MOCK=1.

Step 4.2 With your real key

```
bun run labs/openai/00-check-key.ts
```

Expected output (token numbers will differ slightly)

```
key: sk-proj...Ab3Q  model: gpt-4o-mini
reply: "OK"  (900 ms)
tokens: input=14 output=1
Key works. You can run the other labs without MOCK=1.
```

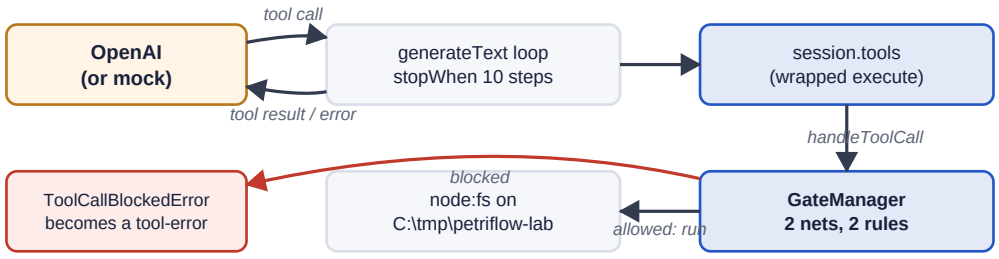
WHAT IS EXECUTING

One `generateText` call with no tools. The usage line is the real token count OpenAI billed. Error branches: 401 wrong key, 429 rate limit or no credit, 404 model not available to your account.

Part 5. The four labs

Lab 1: a file-cleanup agent

Two rules, five tools, one scratch folder. The model is asked to do something the rules forbid (delete before backup, and use rm). Watch it get blocked, read the reason, and recover.



The model never sees an unguarded tool. A blocked call comes back as an error message it can read and recover from.

Lab 1 data flow. generateText loops until the model answers with text or 10 steps pass.

Step 5.1 Rehearse, then run for real

```
MOCK=1 bun run labs/openai/01-file-agent.ts     # free rehearsal
bun run labs/openai/01-file-agent.ts            # real model
```

Expected output in MOCK mode (real mode: same shape, the model's own words and order)

```
=== Lab 1: file cleanup agent ===
mode: MOCK (scripted model, no network)

work folder: C:/tmp/petriflow-lab
rules verified: require-backup-before-delete=3 states, block-rm=2 states
  [gate] allow listFiles
  [gate] BLOCKED delete -> delete requires a successful call to backup first.
  [gate] allow backup
  [gate] allow delete
  [gate] BLOCKED rm -> rm is blocked and cannot be called.

--- Steps ---
1. model called listFiles({})
   -> result: {"ok":true,"files":["old-backup.tar.gz","temp.log"]}
2. model called delete({"name":"temp.log"})
   -> ERROR returned to model: Tool 'delete' blocked: delete requires a successful call to b
    ackup first.
3. model called backup({"name":"temp.log"})
   -> result: {"ok":true,"backedUp":"temp.log.bak"}
4. model called delete({"name":"temp.log"})
   -> result: {"ok":true,"deleted":"temp.log"}
5. model called rm({"name":"old-backup.tar.gz"})
   -> ERROR returned to model: Tool 'rm' blocked: rm is blocked and cannot be called.
   model said: Deleted temp.log after backing it up. I cannot use rm: it is blocked by polic
    y.

--- Gate summary ---
blocked calls: 2
final net state: require-backup-before-delete: ready:1 | block-rm: ready:1
files left: [ "old-backup.tar.gz", "temp.log.bak" ]
```

WHAT IS EXECUTING

`compile()` turns the two rule lines into two nets. `createPetriflowGate` is given `isToolResultError` so a tool returning {ok: false} counts as a failure (a failed backup must not unlock delete). `wrapTools` replaces each tool's execute. When the model calls delete first, `handleToolCall` classifies it as blocked, the wrapper throws `ToolCallBlockedError`, and the AI SDK converts that into a tool-error part that goes back to the model as the next message. A real model reads 'delete requires a successful call to backup first' and calls backup. The listFiles and readFile tools are free because no rule names them.

IN SIMPLE WORDS What to look for in real mode

The real model may skip the mistake and call backup first, because the system prompt (session.systemPrompt()) already lists the rules. Try removing that line from the system prompt in the script: the model then usually tries delete first and the block message teaches it. That is the 0.3.0 design decision from the first guide in action.

Lab 2: a deployment agent with human approval

Four rules, including a human gate that asks **you** in the terminal, and a session limit of two deploys. The task asks for three deploys on purpose.



The sequence nets reset after every deploy, so lint and test must run again each time. The limit net never resets within a session.

The expected path. Every deploy needs a fresh lint and test, an approval, and one unit of the budget.

Step 5.2 Rehearse with auto-approval, then run for real and answer y/n yourself

```
MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
bun run labs/openai/02-deploy-agent.ts            # real: type y or n when asked
FAIL_TESTS=1 bun run labs/openai/02-deploy-agent.ts  # variant: tests fail, \
  deploy stays locked
```

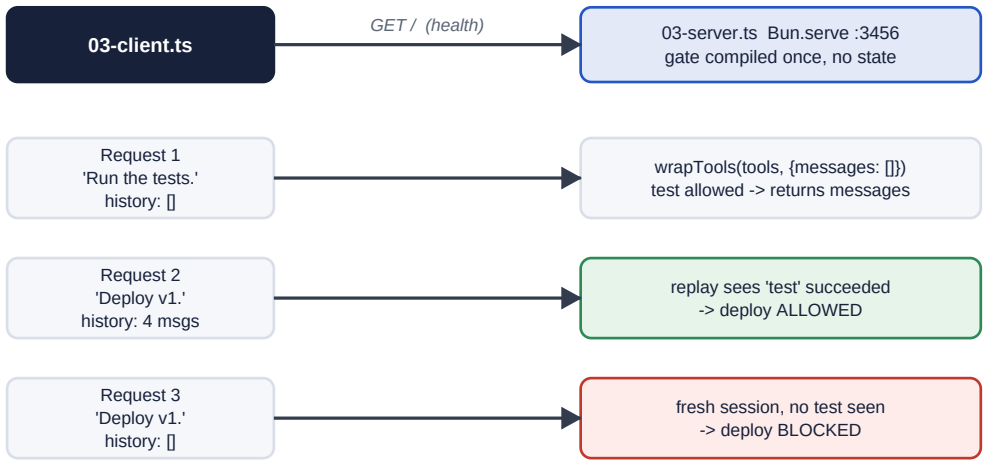
Expected output in MOCK mode (abridged)

```
rules verified: require-lint-before-test=3, require-test-before-deploy=3, approve-before-dep
    loy=2, limit-deploy-2=4
  [gate] BLOCKED deploy(production) -> deploy requires a successful call to test first.
  [gate] allow lint
  [gate] allow test
  [human] Approve: approve (approve-before-deploy) -> auto-approved (AUTO_APPROVE=1)
  [gate] allow deploy(production)
  [gate] allow lint
  [gate] allow test
  [human] Approve: approve (approve-before-deploy) -> auto-approved (AUTO_APPROVE=1)
  [gate] allow deploy(staging)
  [gate] allow lint
  [gate] allow test
  [gate] BLOCKED deploy(production) -> deploy has reached its limit of 2 calls per session.
...
deployments that actually happened: [ "production", "staging" ]
final net state: require-lint-before-test: ready:1 | require-test-before-deploy: gate:1 | ap
    prove-before-deploy: ready:1 | limit-deploy-2: ready:1
```

> **WHAT IS EXECUTING**
>
> Four nets vote on every call. For the first deploy, 'require test before deploy' says blocked and that ends it. After lint and test succeed (deferred transitions, fired on {passed: true}), the next deploy is *gated* by three nets at once: the sequence net, the approval net (type manual, so `confirm()` runs and your terminal asks y/n), and the limit net (one budget token). All must agree; then all three fire in phase 4. The third deploy finds the limit net with an empty budget cup. In real mode answer **n** once to see 'rejected by human review' and watch the model react.

Lab 3: a localhost server where history is the memory

This is the server-integration tutorial from the docs, rewritten with Bun's built-in HTTP server so no extra dependency is needed. The gate is compiled once; every request gets a fresh session; the chat history carries the state.



Three requests from the client. Only the one that carries the history gets to deploy.

Step 5.3 Start the server in one window, run the client in another

```
MOCK=1 bun run labs/openai/03-server.ts        # window 1 (drop MOCK=1 for \
  real; export the key there)
```

Leave it running. Ctrl+C stops it.

```
curl http://localhost:3456/
bun run labs/openai/03-client.ts               # window 2
```

Expected client output in MOCK mode

```
server: {"ok":true,"rules":[{"name":"require-test-before-deploy","reachableStates":3},{"name
    ":"limit-deploy-2","reachableStates":4}],"mode":"mock"}

### Request 1
> Run the tests.   (sending 0 history messages)
model: Tests passed.
gate state: require-test-before-deploy: gate:1 | limit-deploy-2: ready:1, budget:2

### Request 2 (with history)
> Deploy version v1.   (sending 4 history messages)
model: Deploy attempted.
gate state: require-test-before-deploy: ready:1 | limit-deploy-2: ready:1, budget:1

### Request 3 (no history)
> Deploy version v1.   (sending 0 history messages)
model: Deploy attempted.
BLOCKED: Tool 'deploy' blocked: deploy requires a successful call to test first.
gate state: require-test-before-deploy: ready:1 | limit-deploy-2: ready:1, budget:2

Lesson: the server keeps nothing between requests. The chat history is the gate's memory.
```

WHAT IS EXECUTING

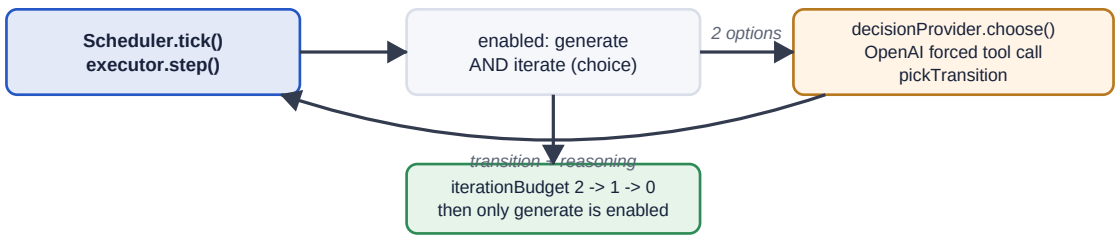
On each POST /chat the server calls `gate.wrapTools(tools, { messages: history })`. That creates a new GateManager and immediately replays the history: `extractReplayEntries` finds the successful 'test' tool result in request 1's messages and fires the matching transition, so request 2 starts with the marking gate:1 and deploy is allowed. Request 3 sends no history, so replay has nothing to do and deploy is blocked. The response includes the full message list so a client can continue the conversation. Watch the server window: it prints 'state after replay' for each request. Security note from the repo's own code comment: in production, keep the history on the server. A client that can send arbitrary history can forge a 'test passed' result.

Raw curl, if you prefer it to the client script:

```
curl -s -X POST http://localhost:3456/chat -H 'content-type: application/json' \
  -d '{"prompt":"Run the tests."}'
```

Lab 4: the model as the engine's decision maker

The other half of the repo. The simple-agent workflow has a choice point: after tool results arrive, both 'generate' and 'iterate' are enabled. A decision provider picks. Here that provider is OpenAI, forced to answer through a tool call so the answer is always one of the enabled names.



The engine asks the model only at real choice points. Deterministic steps never cost a call.

Step 5.4 Run it

```
MOCK=1 bun run labs/openai/04-decision-provider.ts
bun run labs/openai/04-decision-provider.ts
```

Expected output in MOCK mode (real mode: the model's reasoning text appears after the colon)

```
fired: plan
fired: dispatchTool
fired: completeTool
  [llm] chose iterate from [generate, iterate]: mock: budget=2, options=generate/iterate
fired: iterate
fired: plan
fired: dispatchTool
fired: completeTool
  [llm] chose iterate from [generate, iterate]: mock: budget=1, options=generate/iterate
fired: iterate
fired: plan
fired: dispatchTool
fired: completeTool
fired: generate
agent-llm completed

final: {"marking":{...,"responseGenerated":1,"iterationBudget":0},"iteration":2,"status":"co
    mpleted"}
```

WHAT IS EXECUTING

`createExecutor(definition, { decisionProvider })` is the only change from running the workflow plainly. In `step()`, when more than one transition is enabled, the provider's `choose()` is awaited. The OpenAI provider calls generateText with a single tool 'pickTransition' whose schema is an enum of the enabled names and `toolChoice` forces that tool, so the model cannot answer with free text. Notice the third round: the budget is 0, iterate is no longer enabled, only generate is, so the engine does not call the model at all. That is the 'bounded iterations' proof from BENCHMARK.md, felt at runtime. The scheduler's onDecision event prints the model's reasoning.

Part 6. If you have an Anthropic key instead

The repo's own examples run unchanged with an Anthropic key, with two Windows fixes. The same policy questions from Part 1 apply.

```
export ANTHROPIC_API_KEY=sk-ant-...
mkdir -p /c/tmp/project && echo hello > /c/tmp/project/temp.log && echo old > \
  /c/tmp/project/old-backup.tar.gz
FIX='s|new URL(\(.*\), import.meta.url).pathname|Bun.fileURLToPath(new URL(\1, import.meta.url))|'
sed -i "$FIX" examples/0*/agent.ts
bun run examples/01-file-management/agent.ts
bun run examples/02-deployment/agent.ts        # warning: its 'test' tool runs the full repo \
  test suite (20 s per call)
bun run workflows/simple-agent/run-llm.ts
git checkout -- examples                        # undo the path fix when done
```

WHAT IS EXECUTING

Same flow as Labs 1, 2 and 4 but with `anthropic("claude-sonnet-4-5-20250929")` as the model. Example 02's lint and test tools shell out to `bun run check` and `bun test` on the real repo, which is slow and, on Windows, reports failures unless the lab-guide patches are applied, in which case deploy stays locked because test returned passed: false. That is correct behaviour, not a bug.

Part 7. Troubleshooting, cost control, cleanup

7.1 Problems and fixes

Symptom	Meaning	Fix
401 Incorrect API key	Key wrong, revoked, or has a stray space	Re-export the key. With a dummy key this is the expected result of step 4.1.
429 insufficient_quota	No prepaid credit or limit hit	Add credit or raise the limit on the OpenAI dashboard; wait a minute for rate limits.
404 model not found	Model id not enabled for your account	export OPENAI_MODEL=gpt-4o-mini
fetch failed / ENOTFOUND / ETIMEDOUT	Proxy or firewall	export HTTPS_PROXY=http://proxy:port; if it still fails, the network is blocked: use MOCK=1.
Company gateway instead of api.openai.com	Internal endpoint	export OPENAI_BASE_URL=https://your-gateway/v1 (the provider reads it) and use the key they give you.
Cannot find module '@ai-sdk/openai'	Step 3.1 not done	bun add @ai-sdk/openai@^3 at the repo root
Model loops calling blocked tools	Weak model or missing rules in the system prompt	Keep session.systemPrompt() in the system message; the stopWhen cap ends the run anyway.
Lab 2 hangs at 'Approve? (y/n)'	It is waiting for you	Type y or n and press Enter, or run with AUTO_APPROVE=1.
EADDRINUSE on :3456	Server already running	PORT=3457 for the server and BASE=http://localhost:3457 for the client.
ENOENT C:\tmp\petriflow-lab	Folder removed mid-run	The script recreates it on the next start.

7.2 Cost control

- Set a hard monthly limit on the OpenAI account before the first real run.
- Keep gpt-4o-mini unless you are studying model quality. The gate behaves identically with any model.
- Every lab has a step cap (10, 16, 6 per request, 40 ticks). A single run cannot exceed roughly a dozen calls.
- Lab 0 prints token counts. Multiply by the per-token price on the pricing page to know your exact spend.

7.3 Cleanup

```
unset OPENAI_API_KEY                        # or just close the Git Bash window
rm -rf /c/tmp/petriflow-lab
git checkout -- package.json bun.lock       # removes the @ai-sdk/openai line
rm -rf labs/openai                          # if you do not want to keep the labs
git status --short                          # should print nothing
# then revoke the key on https://platform.openai.com/api-keys
```

7.4 Cheat sheet

```
# setup (once)
bun add @ai-sdk/openai@^3
mkdir -p labs/openai            # paste the Appendix files

# rehearse (no key, no network)
MOCK=1 bun run labs/openai/01-file-agent.ts
MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
MOCK=1 bun run labs/openai/03-server.ts        # window 1
bun run labs/openai/03-client.ts               # window 2
MOCK=1 bun run labs/openai/04-decision-provider.ts

# reachability without spending
OPENAI_API_KEY=sk-test-dummy bun run labs/openai/00-check-key.ts   # expect 401

# real (after Part 1 is settled)
export OPENAI_API_KEY=sk-...
bun run labs/openai/00-check-key.ts
bun run labs/openai/01-file-agent.ts
bun run labs/openai/02-deploy-agent.ts
bun run labs/openai/03-server.ts               # window 1 (export the key \
  there too)
bun run labs/openai/03-client.ts               # window 2
bun run labs/openai/04-decision-provider.ts

# cleanup
unset OPENAI_API_KEY; rm -rf /c/tmp/petriflow-lab; git checkout -- \
  package.json bun.lock
```

Appendix. The lab files, paste-ready

Run `mkdir -p labs/openai` first. Then paste each block into Git Bash as one unit, from the `cat` line to the `EOF` line. The files are also in the README table below.

labs/openai/_model.ts

Shared helper: real or mock model, step printer, error explainer.

```
cat > labs/openai/_model.ts <<'EOF'
// Shared helper for the OpenAI labs.
//
//   real mode : needs OPENAI_API_KEY. Model from OPENAI_MODEL (default gpt-4o-mini).
//   mock mode : MOCK=1. A scripted fake model that returns the tool calls you give it.
//               No network, no key, no cost. Use it to rehearse a lab before paying.
import { createOpenAI } from "@ai-sdk/openai";
import { MockLanguageModelV3 } from "ai/test";
import type { LanguageModel } from "ai";
export const MOCK = process.env.MOCK === "1";
export const MODEL_ID = process.env.OPENAI_MODEL ?? "gpt-4o-mini";
export type PlanStep =
  | { tool: string; input?: Record<string, unknown> }
  | { text: string };
function usage(input: number, output: number) {
  return {
    inputTokens: { total: input, noCache: input, cacheRead: 0, cacheWrite: 0 },
    outputTokens: { total: output, text: output, reasoning: 0 },
    raw: undefined,
  };
}
/** A fake model that replays a fixed plan: one step per model call. */
export function scriptedModel(plan: PlanStep[]): LanguageModel {
  let i = 0;
  return new MockLanguageModelV3({
    provider: "mock",
    modelId: "scripted",
    doGenerate: async () => {
      const step = plan[Math.min(i, plan.length - 1)]!;
      i++;
      if ("text" in step) {
        return {
          content: [{ type: "text", text: step.text }],
          finishReason: { unified: "stop", raw: undefined },
          usage: usage(0, 0),
          warnings: [],
        };
      }
      return {
        content: [{
          type: "tool-call",
          toolCallId: `mock-${i}`,
          toolName: step.tool,
          input: JSON.stringify(step.input ?? {}),
        }],
        finishReason: { unified: "tool-calls", raw: undefined },
        usage: usage(0, 0),
        warnings: [],
      };
    },
  }) as unknown as LanguageModel;
}
/** Real OpenAI model, or the scripted mock when MOCK=1. */
export function getModel(plan: PlanStep[]): LanguageModel {
  if (MOCK) return scriptedModel(plan);
  const key = process.env.OPENAI_API_KEY;
  if (!key) {
    console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1 to rehearse.");
    process.exit(1);
  }
  return createOpenAI({ apiKey: key })(MODEL_ID);
```

```
}
export function banner(title: string) {
  console.log(`=== ${title} ===`);
  console.log(MOCK
    ? "mode: MOCK (scripted model, no network)"
    : `mode: REAL (OpenAI ${MODEL_ID})`);
  console.log("");
}
/** Print every step of a generateText result: tool calls, tool results, text. */
export function printSteps(result: { steps: any[]; text: string; usage?: any }) {
  console.log("\n--- Steps ---");
  result.steps.forEach((s, n) => {
    const parts: any[] = s.content ?? [];
    for (const p of parts) {
      if (p.type === "tool-call") {
        console.log(`${n + 1}. model called ${p.toolName}(${JSON.stringify(p.input)})`);
      } else if (p.type === "tool-result") {
        console.log(`   -> result: ${JSON.stringify(p.output).slice(0, 120)}`);
      } else if (p.type === "tool-error") {
        const msg = String(p.error?.message ?? p.error).slice(0, 140);
        console.log(`   -> ERROR returned to model: ${msg}`);
      } else if (p.type === "text" && p.text) {
        console.log(`   model said: ${p.text.slice(0, 200)}`);
      }
    }
  });
  console.log("\n--- Final answer ---");
  console.log(result.text || "(no text)");
  if (result.usage) {
    const u = result.usage;
    console.log(`\ntokens: input=${u.inputTokens ?? "?"} output=${u.outputTokens ?? "?"}`);
  }
}
/** Friendly explanation of common OpenAI errors. */
export function explainError(e: any) {
  const status = e?.statusCode;
  const msg = String(e?.message ?? e);
  console.error("\nERROR:", msg.slice(0, 300));
  if (status === 401) {
    console.error("=> The key is wrong or revoked. Check OPENAI_API_KEY.");
  } else if (status === 429) {
    console.error("=> Rate limit or no credit left (insufficient_quota). Check billing.");
  } else if (status === 404) {
    console.error(`=> Model '${MODEL_ID}' not found. Try OPENAI_MODEL=gpt-4o-mini.`);
  } else if (/ENOTFOUND|ECONNRESET|ETIMEDOUT|fetch failed/i.test(msg)) {
    console.error("=> Network. Set HTTPS_PROXY if your company uses a proxy.");
  }
}
EOF
```

Paste everything including the last EOF line.

labs/openai/00-check-key.ts

Lab 0: one tiny call.

```
cat > labs/openai/00-check-key.ts <<'EOF'
// Step 0: prove the key works and see how many tokens a tiny call costs.
//   OPENAI_API_KEY=sk-... bun run labs/openai/00-check-key.ts
import { generateText } from "ai";
import { createOpenAI } from "@ai-sdk/openai";
import { MODEL_ID, explainError } from "./_model";
const key = process.env.OPENAI_API_KEY;
if (!key) {
  console.error("OPENAI_API_KEY is not set. In Git Bash:  export OPENAI_API_KEY=sk-...");
  process.exit(1);
}
console.log(`key: ${key.slice(0, 7)}...${key.slice(-4)}  model: ${MODEL_ID}`);
try {
  const t0 = Date.now();
  const r = await generateText({
    model: createOpenAI({ apiKey: key })(MODEL_ID),
```

```
    prompt: "Reply with exactly the two letters: OK",
  });
  console.log(`reply: ${JSON.stringify(r.text)}  (${Date.now() - t0} ms)`);
  console.log(`tokens: input=${r.usage.inputTokens} output=${r.usage.outputTokens}`);
  console.log("Key works. You can run the other labs without MOCK=1.");
} catch (e) {
  explainError(e);
  process.exit(1);
}
EOF
```

Paste everything including the last EOF line.

labs/openai/01-file-agent.ts

Lab 1: file cleanup agent.

```
cat > labs/openai/01-file-agent.ts <<'EOF'
// Lab 1: a file-cleanup agent with two rules, driven by OpenAI (or the mock).
//   rehearse:  MOCK=1 bun run labs/openai/01-file-agent.ts
//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/01-file-agent.ts
import { generateText, tool, stepCountIs } from "ai";
import { z } from "zod";
import * as fs from "node:fs/promises";
import { join } from "node:path";
import { compile } from "@petriflow/rules";
import { createPetriflowGate } from "@petriflow/vercel-ai";
import { getModel, banner, printSteps, explainError } from "./_model";
banner("Lab 1: file cleanup agent");
// A scratch folder with two files. Nothing outside it is touched.
const WORKDIR = process.env.WORKDIR ?? "C:/tmp/petriflow-lab";
await fs.rm(WORKDIR, { recursive: true, force: true }); // start clean every run
await fs.mkdir(WORKDIR, { recursive: true });
await fs.writeFile(join(WORKDIR, "temp.log"), "temporary log\n");
await fs.writeFile(join(WORKDIR, "old-backup.tar.gz"), "pretend archive\n");
console.log(`work folder: ${WORKDIR}`);
// 1. Rules -> verified nets.
const { nets, verification } = compile(`
  require backup before delete
  block rm
`);
console.log("rules verified:",
  verification.map((v) => `${v.name}=${v.reachableStates} states`).join(", "));
// 2. The gate. Tools that return { ok: false } count as failures.
const blocked: string[] = [];
const gate = createPetriflowGate(nets, {
  isToolResultError: (_name, result) => (result as any)?.ok === false,
  onDecision: (event, decision) => {
    if (decision?.block) {
      blocked.push(`${event.toolName}: ${decision.reason}`);
      console.log(`  [gate] BLOCKED ${event.toolName} -> ${decision.reason}`);
    } else {
      console.log(`  [gate] allow ${event.toolName}`);
    }
  },
});
// 3. Tools, wrapped. listFiles and readFile are free because no rule names them.
const session = gate.wrapTools({
  listFiles: tool({
    description: "List files in the work folder",
    inputSchema: z.object({}),
    execute: async () => ({ ok: true, files: await fs.readdir(WORKDIR) }),
  }),
  readFile: tool({
    description: "Read a file in the work folder",
    inputSchema: z.object({ name: z.string() }),
    execute: async ({ name }) => {
      const content = await fs.readFile(join(WORKDIR, name), "utf-8");
      return { ok: true, content };
    },
  }),
  backup: tool({
```

```
    description: "Copy a file to <name>.bak before changing it",
    inputSchema: z.object({ name: z.string() }),
    execute: async ({ name }) => {
      await fs.copyFile(join(WORKDIR, name), join(WORKDIR, name + ".bak"));
      return { ok: true, backedUp: name + ".bak" };
    },
  }),
  delete: tool({
    description: "Delete a file (a backup must succeed first)",
    inputSchema: z.object({ name: z.string() }),
    execute: async ({ name }) => {
      await fs.unlink(join(WORKDIR, name));
      return { ok: true, deleted: name };
    },
  }),
  rm: tool({
    description: "Force-remove a file",
    inputSchema: z.object({ name: z.string() }),
    execute: async ({ name }) => {
      await fs.rm(join(WORKDIR, name), { force: true });
      return { ok: true, removed: name };
    },
  }),
});
// 4. What the mock model does if MOCK=1 (a realistic path, including one mistake).
const model = getModel([
  { tool: "listFiles" },
  { tool: "delete", input: { name: "temp.log" } },      // blocked: no backup yet
  { tool: "backup", input: { name: "temp.log" } },
  { tool: "delete", input: { name: "temp.log" } },      // allowed now
  { tool: "rm", input: { name: "old-backup.tar.gz" } }, // always blocked
  { text: "Deleted temp.log after backing it up. "
        + "I cannot use rm: it is blocked by policy." },
]);
// 5. Run the agent loop.
const SYSTEM = "You are a careful file-cleanup agent. Use the tools. "
  + "If a tool is blocked, read the reason and adapt.\n\n" + session.systemPrompt();
const PROMPT = "Clean up the work folder: list the files, delete temp.log, "
  + "and rm old-backup.tar.gz.";
try {
  const result = await generateText({
    model,
    system: SYSTEM,
    tools: session.tools,
    stopWhen: stepCountIs(10),
    prompt: PROMPT,
  });
  printSteps(result);
  console.log("\n--- Gate summary ---");
  console.log(`blocked calls: ${blocked.length}`);
  for (const b of blocked) console.log("  " + b);
  console.log("final net state:", session.formatStatus().replace(/\n/g, " | "));
  console.log("files left:", await fs.readdir(WORKDIR));
} catch (e) {
  explainError(e);
  process.exit(1);
}
EOF
```

Paste everything including the last EOF line.

labs/openai/02-deploy-agent.ts

Lab 2: deployment agent with approvals and a limit.

```
cat > labs/openai/02-deploy-agent.ts <<'EOF'
// Lab 2: a deployment agent with chained rules, a real human approval prompt, and a limit.
//   rehearse:  MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/02-deploy-agent.ts   (type y/n)
//   variant:   FAIL_TESTS=1 ...  makes the test tool fail, so deploy stays locked.
import { generateText, tool, stepCountIs } from "ai";
import { z } from "zod";
```

```
import { createInterface } from "node:readline";
import { compile } from "@petriflow/rules";
import { createPetriflowGate } from "@petriflow/vercel-ai";
import { getModel, banner, printSteps, explainError } from "./_model";
banner("Lab 2: deployment agent");
const { nets, verification } = compile(`
  require lint before test
  require test before deploy
  require human-approval before deploy
  limit deploy to 2 per session
`);
console.log("rules verified:",
  verification.map((v) => `${v.name}=${v.reachableStates}`).join(", "));
// Human approval: a terminal prompt, unless AUTO_APPROVE=1.
async function askApproval(title: string, message: string): Promise<boolean> {
  if (process.env.AUTO_APPROVE === "1") {
    console.log(`  [human] ${title} -> auto-approved (AUTO_APPROVE=1)`);
    return true;
  }
  const rl = createInterface({ input: process.stdin, output: process.stdout });
  return new Promise((resolve) => {
    rl.question(`\n  [human] ${title}\n  ${message}\n  Approve? (y/n) `, (a) => {
      rl.close();
      resolve(a.trim().toLowerCase() === "y");
    });
  });
}
const gate = createPetriflowGate(nets, {
  isToolResultError: (_name, result) => (result as any)?.passed === false,
  confirm: askApproval,
  onDecision: (event, decision) => {
    const target = (event.input as any)?.environment;
    const env = target ? `(${target})` : "";
    console.log(decision?.block
      ? `  [gate] BLOCKED ${event.toolName}${env} -> ${decision.reason}`
      : `  [gate] allow ${event.toolName}${env}`);
  },
});
// Simulated pipeline. Nothing real is deployed anywhere.
const deployed: string[] = [];
const session = gate.wrapTools({
  lint: tool({
    description: "Run the linter",
    inputSchema: z.object({}),
    execute: async () => ({ passed: true, warnings: 0 }),
  }),
  test: tool({
    description: "Run the test suite",
    inputSchema: z.object({}),
    execute: async () => {
      const passed = process.env.FAIL_TESTS !== "1";
      return { passed, failed: passed ? 0 : 3 };
    },
  }),
  deploy: tool({
    description: "Deploy the current build to an environment",
    inputSchema: z.object({ environment: z.enum(["production", "staging"]) }),
    execute: async ({ environment }) => {
      deployed.push(environment);
      return { deployed: true, environment, version: `1.${deployed.length}.0` };
    },
  }),
  checkStatus: tool({
    description: "Check what is deployed where",
    inputSchema: z.object({}),
    execute: async () => ({ deployed }),
  }),
});
// Mock plan: tries to skip steps, gets corrected, hits the limit on the third deploy.
const model = getModel([
  { tool: "deploy", input: { environment: "production" } },  // blocked: no test
  { tool: "lint" },
  { tool: "test" },
```

```
  { tool: "deploy", input: { environment: "production" } },  // approval, then allowed
  { tool: "lint" },
  { tool: "test" },
  { tool: "deploy", input: { environment: "staging" } },     // allowed (2 of 2)
  { tool: "lint" },
  { tool: "test" },
  { tool: "deploy", input: { environment: "production" } },  // blocked: limit
  { text: "Deployed to production and staging. "
        + "A third deploy is blocked by the session limit." },
]);
const SYSTEM = "You are a deployment agent. Follow the pipeline rules. "
  + "If a tool is blocked, read the reason and do what it says.\n\n"
  + session.systemPrompt();
const PROMPT = "Deploy to production, then deploy to staging, "
  + "then deploy to production once more.";
try {
  const result = await generateText({
    model,
    system: SYSTEM,
    tools: session.tools,
    stopWhen: stepCountIs(16),
    prompt: PROMPT,
  });
  printSteps(result);
  console.log("\ndeployments that actually happened:", deployed);
  console.log("final net state:", session.formatStatus().replace(/\n/g, " | "));
} catch (e) {
  explainError(e);
  process.exit(1);
}
EOF
```

Paste everything including the last EOF line.

labs/openai/03-server.ts

Lab 3: localhost server with replay.

```
cat > labs/openai/03-server.ts <<'EOF'
// Lab 3: a localhost chat server. The gate is stateless; state comes from message history.
//   rehearse:  MOCK=1 bun run labs/openai/03-server.ts
//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/03-server.ts
// Then, in a second Git Bash window:  bun run labs/openai/03-client.ts
import { generateText, tool, stepCountIs } from "ai";
import type { ModelMessage } from "ai";
import { z } from "zod";
import { compile } from "@petriflow/rules";
import { createPetriflowGate } from "@petriflow/vercel-ai";
import { createOpenAI } from "@ai-sdk/openai";
import { MOCK, MODEL_ID, scriptedModel, banner } from "./_model";
banner("Lab 3: localhost server with replay");
const PORT = Number(process.env.PORT ?? 3456);
// 1. Compile once at startup. The gate object holds no per-user state.
const { nets, verification } = compile(`
  require test before deploy
  limit deploy to 2 per session
`);
const gate = createPetriflowGate(nets, {
  isToolResultError: (_n, r) => (r as any)?.passed === false,
  onDecision: (event, decision) => {
    console.log(decision?.block
      ? `  [gate] BLOCKED ${event.toolName}: ${decision.reason}`
      : `  [gate] allow ${event.toolName}`);
  },
});
// 2. Tools shared by every request (they are pure, so sharing is safe).
const tools = {
  test: tool({
    description: "Run the test suite",
    inputSchema: z.object({}),
    execute: async () => ({ passed: true }),
  }),
```

```
  deploy: tool({
    description: "Deploy a version to production",
    inputSchema: z.object({ version: z.string() }),
    execute: async ({ version }) => ({ deployed: version }),
  }),
};
// 3. Pick the model. The mock reacts to the last user message.
function modelFor(prompt: string) {
  if (!MOCK) return createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID);
  const p = prompt.toLowerCase();
  if (p.includes("test")) {
    return scriptedModel([{ tool: "test" }, { text: "Tests passed." }]);
  }
  if (p.includes("deploy")) {
    return scriptedModel([
      { tool: "deploy", input: { version: "v1" } },
      { text: "Deploy attempted." },
    ]);
  }
  return scriptedModel([{ text: "I can run tests or deploy." }]);
}
if (!MOCK && !process.env.OPENAI_API_KEY) {
  console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1.");
  process.exit(1);
}
const SYSTEM_PREFIX = "You are a release assistant. Use the tools. "
  + "If a tool is blocked, explain the reason.\n\n";
// 4. The server. POST /chat {prompt, messages?} -> {text, blocked, status, messages}
const server = Bun.serve({
  port: PORT,
  async fetch(req) {
    const url = new URL(req.url);
    if (req.method === "GET" && url.pathname === "/") {
      const mode = MOCK ? "mock" : `openai:${MODEL_ID}`;
      return Response.json({ ok: true, rules: verification, mode });
    }
    if (req.method === "POST" && url.pathname === "/chat") {
      const body = (await req.json()) as { prompt: string; messages?: ModelMessage[] };
      const history: ModelMessage[] = body.messages ?? [];
      console.log(`\nPOST /chat  prompt="${body.prompt}"  history=${history.length}`);
      // Fresh session per request. Given history, replay advances the nets first.
      const session = gate.wrapTools(tools, { messages: history });
      console.log("  state after replay:", session.formatStatus().replace(/\n/g, " | "));
      const blocked: string[] = [];
      const messages: ModelMessage[] = [...history, { role: "user", content: body.prompt }];
      try {
        const result = await generateText({
          model: modelFor(body.prompt),
          system: SYSTEM_PREFIX + session.systemPrompt(),
          tools: session.tools,
          messages,
          stopWhen: stepCountIs(6),
        });
        for (const s of result.steps) {
          for (const p of (s.content ?? []) as any[]) {
            if (p.type === "tool-error") blocked.push(String(p.error?.message ?? p.error));
          }
        }
        return Response.json({
          text: result.text,
          blocked,
          status: session.formatStatus(),
          messages: [...messages, ...result.response.messages],
        });
      } catch (e: any) {
        const error = String(e?.message ?? e);
        return Response.json({ error, status: e?.statusCode ?? 500 }, { status: 500 });
      }
    }
    return new Response("not found", { status: 404 });
  },
});
console.log(`listening on http://localhost:${server.port}`);
```

```
console.log(`try:  curl http://localhost:${server.port}/`);
console.log("then: bun run labs/openai/03-client.ts   (other window). Ctrl+C stops.");
EOF
```

Paste everything including the last EOF line.

labs/openai/03-client.ts

Lab 3: client that sends three requests.

```
cat > labs/openai/03-client.ts <<'EOF'
// Client for Lab 3. Sends three requests and shows why message history matters.
//   bun run labs/openai/03-client.ts            (server must be running on :3456)
export {};
const BASE = process.env.BASE ?? "http://localhost:3456";
async function chat(label: string, prompt: string, messages: unknown[] = []) {
  console.log(`\n### ${label}`);
  console.log(`> ${prompt}   (sending ${messages.length} history messages)`);
  const res = await fetch(`${BASE}/chat`, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({ prompt, messages }),
  });
  const data: any = await res.json();
  if (data.error) {
    console.log("server error:", data.error);
    return data;
  }
  console.log("model:", data.text);
  for (const b of data.blocked ?? []) console.log("BLOCKED:", b);
  console.log("gate state:", String(data.status).replace(/\n/g, " | "));
  return data;
}
const health = await (await fetch(`${BASE}/`)).json();
console.log("server:", JSON.stringify(health));
// Request 1: run the tests. The session for this request ends with the response.
const r1 = await chat("Request 1", "Run the tests.");
// Request 2: deploy WITH the history from request 1.
// Replay sees the successful test, so deploy is allowed.
await chat("Request 2 (with history)", "Deploy version v1.", r1.messages);
// Request 3: deploy WITHOUT history.
// A fresh session has never seen a test, so deploy is blocked.
await chat("Request 3 (no history)", "Deploy version v1.");
console.log("\nLesson: the server keeps nothing between requests. "
  + "The chat history is the gate's memory.");
EOF
```

Paste everything including the last EOF line.

labs/openai/04-decision-provider.ts

Lab 4: OpenAI as the engine's decision provider.

```
cat > labs/openai/04-decision-provider.ts <<'EOF'
// Lab 4: the engine side. OpenAI chooses which transition fires at a choice point.
//   rehearse:  MOCK=1 bun run labs/openai/04-decision-provider.ts
//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/04-decision-provider.ts
import { generateText, tool } from "ai";
import { z } from "zod";
import { Database } from "bun:sqlite";
import { Scheduler, createExecutor, sqliteAdapter } from "@petriflow/engine";
import type { DecisionProvider } from "@petriflow/engine/decision";
import { createOpenAI } from "@ai-sdk/openai";
import { definition } from "../../workflows/simple-agent/definition";
import { MOCK, MODEL_ID, banner, explainError } from "./_model";
banner("Lab 4: OpenAI as the engine's decision provider");
type Provider = DecisionProvider<string, Record<string, unknown>>;
// The mock chooses 'iterate' while budget remains, then 'generate'.
const mockProvider: Provider = {
  async choose(req) {
    const names = req.enabled.map((t) => t.name);
```

```
    const budget = (req.marking as any).iterationBudget ?? 0;
    const pick = names.includes("iterate") && budget > 0 ? "iterate" : names[0]!;
    const reasoning = `mock: budget=${budget}, options=${names.join("/")}`;
    return { transition: pick, reasoning };
  },
};
// The real provider asks OpenAI to pick one enabled transition via a forced tool call.
const openaiProvider: Provider = {
  async choose(req) {
    const names = req.enabled.map((t) => t.name) as [string, ...string[]];
    const marking = Object.entries(req.marking)
      .filter(([, v]) => (v as number) > 0)
      .map(([k, v]) => `${k}=${v}`)
      .join(", ");
    const enabled = req.enabled
      .map((t) => `${t.name} [${t.inputs}] -> [${t.outputs}]`)
      .join("; ");
    const result = await generateText({
      model: createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID),
      system: `You control the workflow "${req.workflowName}". `
        + "Prefer to iterate once to gather more tool results, then generate. "
        + "Pick exactly one transition.",
      prompt: `Marking: ${marking}\nContext: ${JSON.stringify(req.context)}\n`
        + `Enabled: ${enabled}`,
      tools: {
        pickTransition: tool({
          description: "Choose the transition to fire next",
          inputSchema: z.object({ transition: z.enum(names), reasoning: z.string() }),
        }),
      },
      toolChoice: { type: "tool", toolName: "pickTransition" },
    });
    const input = result.toolCalls[0]!.input as any;
    return { transition: input.transition, reasoning: input.reasoning };
  },
};
if (!MOCK && !process.env.OPENAI_API_KEY) {
  console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1.");
  process.exit(1);
}
const db = new Database(":memory:");
const executor = createExecutor(definition as any, {
  decisionProvider: MOCK ? mockProvider : openaiProvider,
});
const scheduler = new Scheduler(executor, { adapter: sqliteAdapter(db, definition.name) }, {
  onDecision: (_id, name, reasoning, candidates) => {
    console.log(`  [llm] chose ${name} from [${candidates.join(", ")}]: ${reasoning}`);
  },
  onFire: (_id, name) => console.log(`fired: ${name}`),
  onComplete: (id) => console.log(`${id} completed`),
  onError: (_id, err) => explainError(err),
});
await scheduler.createInstance("agent-llm");
for (let i = 0; i < 40; i++) {
  const n = await scheduler.tick();
  if (n === 0) break;
}
const state = await scheduler.inspect("agent-llm");
console.log("\nfinal:", JSON.stringify({
  marking: state.marking,
  iteration: (state.context as any).iteration,
  status: state.status,
}));
EOF
```

Paste everything including the last EOF line.

labs/openai/README.md

Index of the folder.

```
cat > labs/openai/README.md <<'EOF'
```

```
# OpenAI labs for PetriFlow
Four runnable agents that drive the PetriFlow gate with an OpenAI model, plus a key check.
Every lab also runs with `MOCK=1`: a scripted fake model, no network, no cost.
Setup (once, at the repo root): `bun add @ai-sdk/openai@^3`
Revert later with `git checkout -- package.json bun.lock`.
Environment variables:
- `OPENAI_API_KEY`  required for real mode
- `OPENAI_MODEL`    default `gpt-4o-mini`
- `MOCK=1`          use the scripted model instead of OpenAI
- `WORKDIR`         lab 1 scratch folder, default `C:/tmp/petriflow-lab`
- `PORT` / `BASE`   lab 3 server port (default 3456) and client base URL
Files:
- `00-check-key.ts`         key works, token usage (real mode only)
- `01-file-agent.ts`        require backup before delete, block rm
- `02-deploy-agent.ts`      lint -> test -> deploy, human approval (y/n), limit 2
- `03-server.ts`            localhost server, per-request sessions, replay from history
- `03-client.ts`            sends three requests to the server
- `04-decision-provider.ts` OpenAI picks transitions inside the workflow engine
Rehearse for free:
- `MOCK=1 bun run labs/openai/01-file-agent.ts`
- `MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts`
- `MOCK=1 bun run labs/openai/03-server.ts` then `bun run labs/openai/03-client.ts`
- `MOCK=1 bun run labs/openai/04-decision-provider.ts`
Real: export `OPENAI_API_KEY`, drop `MOCK=1`, start with `00-check-key.ts`.
EOF
```

Paste everything including the last EOF line.

End of guide.